

Enriching Object-Centric Event Data with Process Scopes: A Framework for Aggregation and Analysis

Shahrzad Khayatbashi¹, Majid Rafiei², Jiayuan Chen², Timotheus Kampik², Gregor Berg², and Amin Jalali³

¹ Linköping University, Linköping, Sweden,
shahrzad.khayatbashi@liu.se

² SAP Signavio, Berlin, Germany,
(firstname.lastname)@sap.com

³ Stockholm University, Stockholm, Sweden,
aj@dsv.su.se

Abstract. Object-Centric Process Mining enables the analysis of complex operational behavior by capturing interactions among multiple business objects (e.g., orders, items, deliveries). These interactions are recorded using Object-Centric Event Data (OCED) formats, such as the Object-Centric Event Log (OCEL). However, existing formats lack explicit definitions of process scopes, which restricts analysis to individual processes and limits insights to a low level of granularity. In practice, OCED often spans multiple interrelated processes, as shared objects connect events across organizational functions. This structure reflects how value is created along the organizational value chain, but introduces challenges for interpretation when process boundaries are not clearly defined. Moreover, process definitions are typically subjective and context-dependent; they vary across organizations, roles, and analytical goals, and cannot always be discovered automatically. To address these challenges, we propose a method for embedding analyst-defined process scopes into OCEL. This enables the structured representation of multiple coexisting processes, supports the aggregation of event data across scopes, and facilitates analysis at varying levels of abstraction. We demonstrate the applicability of our approach using a publicly available OCEL log and provide supporting tools for scope definition and analysis.

Keywords: Object-Centric Process Mining, Object-Centric Event Data, Process Scope Definition, Process Aggregation, Event Log Abstraction

1 Introduction

Object-Centric Process Mining (OCPM) [1] has emerged as a promising paradigm for analyzing complex, real-world business processes [2–5]. While traditional process mining techniques support the discovery, monitoring, and improvement of business processes, they typically rely on the assumption that each event belongs to a single case, such as an order, claim, or ticket. This single-case perspective works well for simple scenarios but often struggles in more realistic contexts, creating challenges such as artificial divergence or convergence of paths, missed interactions, and gaps in understanding how different objects interact across multiple cases [1, 6].

To overcome these limitations, several Object-Centric Event Data (OCED) formats have been introduced [7–9]. Among them, the Object-Centric Event Log (OCEL) [10] is the most widely adopted and forms the foundation for various process mining algorithms, libraries, and tools [11–16]. OCEL supports events that are related to multiple business objects, enabling a more expressive and accurate representation of process behavior. For example, in a sales scenario, a single event might reference an order, a customer, and several items. Fig. 1 illustrates an abstract OCEL where events are connected to one or more objects. This multi-object perspective better reflects how work is executed in practice and allows analysts to examine behavior from different viewpoints, revealing hidden dependencies and overlaps across processes [2–5].

Despite its practically expressive nature, OCEL lacks explicit support for defining process scopes, which are essential for structured and context-sensitive analysis. In Business Process Management (BPM), identifying and organizing processes is a foundational step, often referred to as process identification or process architecture [17]. Although OCED can be extracted directly from operational systems, defining the boundaries of processes requires input from domain experts such as analysts, process owners, or managers, who rely on organizational objectives, responsibilities, and analytical goals.

Fig. 1 illustrates how different stakeholders may define overlapping or nested scopes based on their roles and perspectives. These definitions are inherently subjective yet vital for making sense of complex event data. In the absence of explicit scoping in the log, analysts typically apply manual filtering to isolate relevant subsets. While helpful, this approach is ad hoc and limits the ability to systematically explore how processes interact, overlap, or align with organizational structures.

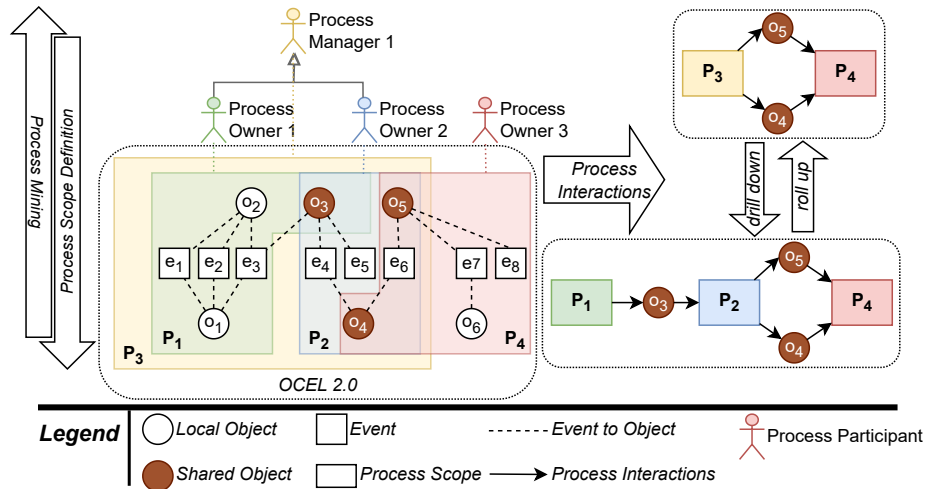


Fig. 1: Example of an Object-Centric Event Log (OCEL) where events connect to multiple objects and are grouped into explicit process scopes (P1–P4). The figure illustrates how shared objects can link processes, enabling the discovery of process-level interactions.

To overcome this limitation, we propose a method for embedding explicit, analyst-defined process scopes into OCEL while preserving full compatibility with the OCEL 2.0 standard [10]. This enables a structured representation of multiple coexisting processes within the same log. Our approach supports both fine-grained and coarse-grained scoping, allowing analysts to define scopes for local subprocesses and aggregate them into higher-level processes. For example, analysts may annotate segments of the event log with scopes P_1 and P_2 , and subsequently define a broader scope P_3 that encompasses both, thereby capturing a higher level of abstraction.

This capability reflects the hierarchical nature of organizational process landscapes, where operational staff focus on local tasks while managers evaluate performance at the strategic level. By explicitly linking low-level and high-level scopes, analysts can traverse between different levels of abstraction. This not only facilitates drill-down and roll-up style analysis but also supports the exploration of value chains, process handovers, and inter-process dependencies.⁴

To operationalize our approach, we provide two supporting toolsets. The first toolset allows analysts to define and embed process scopes into OCELS in a structured and reproducible manner. The second toolset enables the discovery of inter-process interactions within scope-enriched OCELS. It supports the discovery of how processes interact through shared or linked object types, enabling analysts to identify dependencies among processes based on these connections. Together, these tools enhance the analytical power of object-centric process mining by combining expert-driven scoping with scalable, data-driven insights, and by supporting analysis at multiple levels of abstraction.

The remainder of this paper is structured as follows. Section 2 provides a brief background. Section 3 introduces both the formal data model and a grammar for specifying scope rules. Section 4 describes the implementation of the supporting toolsets. Section 5 demonstrates the applicability of our approach using a publicly available OCEL log. Finally, Section 6 summarizes the main contributions and outlines directions for future research.

2 Background

Business Process Management (BPM) has traditionally focused on improving organizational efficiency and effectiveness by analyzing, modeling, and optimizing processes [18]. A foundational activity in BPM is defining the scope of a process, which refers to determining what constitutes a process instance and where its boundaries lie [17]. In process mining, this is typically achieved by selecting a case identifier that guides the extraction of event logs from information systems. As a result, most case-based event logs reflect a single process perspective and omit any representation of inter-process interactions or overlapping responsibilities [6].

In practice, however, processes often do not operate in isolation. They interact, overlap, and co-exist across organizational units, functional domains, and system boundaries. In [19], van der Aalst outlined several types of process interoperability, such as chained execution, subcontracting, case transfer, capacity sharing, and loosely coupled

⁴Note that most of the concepts discussed in this paper are the subject of U.S. Patent Application No. 18/944,997

execution. These patterns highlight the interconnected nature of modern operations and the need for analytical methods capable of representing and exploring such complexity.

While the need to study these interactions was identified early in process mining research [20], traditional techniques continue to rely on a single case notion that limits the ability to analyze process relationships that span across object types or organizational functions. The Object-Centric Event Log (OCEL) [10] was introduced to address single-case process analysis limitations. OCEL supports associating events with multiple related objects, which enables capturing multi-perspective behavior within a single log. This structure has advanced the development of Object-Centric Process Mining (OCPM), which provides tools to analyze processes from various object viewpoints and uncover interactions, dependencies, and parallel execution patterns [21].

As the richness of OCEL increases, so does the complexity of the analysis [2]. To address this, recent work has introduced formal operations for adjusting the level of detail in OCEL. These include drill-down, roll-up, unfold, and fold, which allow analysts to transform logs by grouping or separating object types and event semantics [14, 22]. These operations help adapt the analysis to stakeholder needs, improve model interpretability, and allow switching between fine-grained and high-level perspectives. However, such techniques are primarily data-centric. They modify the structure of the event log to control the level of abstraction but do not offer an explicit mechanism for defining what should be considered as a process. From a BPM standpoint, abstraction involves not only viewing data at different resolutions but also organizing and understanding how multiple processes relate, overlap, or compose a broader process landscape.

In this paper, we extend OCEL by introducing support for explicitly defined process scopes. This allows analysts to embed structured process definitions into the log based on domain knowledge or analytical objectives. Our approach complements prior work on log transformation by enabling not only more flexible analysis, but also a clearer representation of multiple coexisting processes and their relationships. It supports aggregating behavior across process scopes and enables reasoning at multiple levels of abstraction, from local activities to strategic process structures.

3 Approach

To address the lack of explicit process scope definition in object-centric process mining, we propose a structured extension of the OCEL format that allows analysts to define and embed process boundaries directly within the event log. This enhancement enables processes to be treated as first-class entities in OCEL, making their structure and interrelations explicit.

Our approach is grounded in the observation that process definitions in real-world environments are inherently context-dependent and shaped by domain-specific knowledge. As discussed in the background, this subjectivity poses challenges for automated or system-defined scope discovery. To overcome this, we introduce the notion of *scope-enriched OCELS*, which incorporates analyst-defined process scopes into the log using a formal specification language and an associated filtering mechanism. This enrichment renders process boundaries transparent, reproducible, and machine-interpretable, thereby enabling structured analysis of coexisting processes and their interactions.

3.1 Formal definitions

To formalize our approach, we begin by defining the foundational elements of the OCEL data model. Our formulation adheres to the OCEL 2.0 specification [10], while introducing targeted extensions to support the representation of explicit process scopes. These extensions provide the structural basis for embedding scope definitions within the event log and for enabling scoped analysis over object-centric event data.

Definition 1. We define the following **universes** [9, 10]:

$\mathbb{U}_{eid}$ is the universe of event identifiers, $\mathbb{U}_{etype}$ is the universe of event types, $\mathbb{U}_{oid}$ is the universe of object identifiers, $\mathbb{U}_{qual}$ is the universe of qualifiers, $\mathbb{U}_{time}$ is the universe of timestamps, $\mathbb{U}_{val}$ is the universe of attribute values, $\mathbb{U}_{att}$ is the universe of attribute names, and $\mathbb{U}_{otype}$ is the universe of object types, with $process \in \mathbb{U}_{otype}$. We assume that $\mathbb{U}_{time}$ has a smallest element 0 and a largest element ∞ . We also define an object type $process$ to distinguish objects with the type of process, where the object identifier can refer to the name of the process.

The OCEL structure itself is then defined as a tuple that includes events, objects, their associated attributes, and their qualified relations. This formalism provides the foundation upon which we later build the notion of enriched process scopes.

Definition 2. An **Object-Centric Event Log (OCEL)** L is a tuple $(E, O, EA, OA, evtype, evid, time, objtype, objid, eatype, oatype, eaval, oaval, E2O, O2O)$ [9,10], where:

- E and O are sets of events and sets of objects, where $E \cap O = \emptyset$,
- $EA \subseteq \mathbb{U}_{att}$ and $OA \subseteq \mathbb{U}_{att}$ are sets of attributes for events and objects, respectively,
- $evtype : E \rightarrow \mathbb{U}_{etype}$ is a function assigning event types to events,
- $evid : E \rightarrow \mathbb{U}_{eid}$ is a function assigning event identifier to events,
- $etime : E \rightarrow \mathbb{U}_{time}$ is a function assigning timestamps to events,
- $objtype : O \rightarrow \mathbb{U}_{otype}$ is a function assigning object types to objects,
- $objid : O \rightarrow \mathbb{U}_{oid}$ is a function assigning object identifier to objects,
- $eatype : EA \rightarrow \mathbb{U}_{etype}$ is a function assigning event types to event attributes,
- $oatype : OA \rightarrow \mathbb{U}_{otype}$ is a function assigning object types to object attributes,
- $eaval : (E \times EA) \rightarrow \mathbb{U}_{val}$ is a partial function assigning values to (some) event attributes such that $evtype(e) = eatype(ea)$ for all $(e, ea) \in \text{dom}(eaval)$,
- $oaval : (O \times OA \times \mathbb{U}_{time}) \rightarrow \mathbb{U}_{val}$ assigns values to object attributes such that $objtype(o) = oatype(oa)$ for all $(o, oa, t) \in \text{dom}(oaval)$,
- $E2O \subseteq E \times \mathbb{U}_{qual} \times O$ are the qualified event-to-object relations, and
- $O2O \subseteq O \times \mathbb{U}_{qual} \times O$ are the qualified object-to-object relations.

For any partial function $f : D \rightarrow R$, if there exists $d \in D$ where $d \notin \text{dom}(f)$, we say $f(d) = \perp$. We gave an excerpt of the OCEL 2.0 definition that we needed in this paper. We refer the readers to [10] for the full specification.

To introduce process scoping explicitly, we define a *scope-enriched OCEL* (denoted POCEL) as an OCEL that contains objects of type `process`, each representing a specific process scope.

Definition 3. A **Scope-enriched OCEL (POCEL)** is a specific type of OCEL $(E, O, EA, OA, evtype, evid, time, objtype, objid, eatype, oatype, eaval, oaval, E2O, O2O)$, where there exists $o, o' \in O$, $e \in E$, $q, q' \in \mathbb{U}_{qual}$ such that $objtype(o) = process$ and $(e, q, o) \in E2O$ and $o \neq o'$ and $(o, q', o') \in O2O$.

3.2 The enrichment language

To enable the construction of scope-enriched OCEs in a repeatable and domain-sensitive manner, we introduce a formal specification mechanism, referred to as the *Enrichment language*. This grammar-based language allows analysts to define process scopes by formulating inclusion and exclusion criteria over event and object attributes, object types, and event types. The language is designed to balance expressive power with ease of use, supporting the specification of complex scoping logic through structured and interpretable rule compositions.

The syntax of the language is defined as a context-free grammar. Terminal symbols are drawn from the universe of event and object attributes, object and event type labels, constant values, and logical operators. The grammar supports the construction of filter statements and compound conditions, including conjunctions, disjunctions, and nested expressions, enabling flexible and fine-grained scope definitions.

Definition 4 (The Enrichment Language Grammar). *We define the enrichment language as a context-free grammar $G = (\Sigma, N, R, S)$, where:*

- Σ is the set of terminal symbols, i.e., $\Sigma = \{INCLUDE, EXCLUDE, AND, OR\} \cup \mathcal{P}(\mathbb{U}_{otype}) \cup \mathcal{P}(\mathbb{U}_{etype}) \cup \mathbb{U}_{otype} \cup \mathbb{U}_{etype} \cup \mathbb{U}_{att} \cup \mathbb{U}_{val}$.
- N is the set of non-terminal symbols, i.e., $N = \{\langle ruleset \rangle, \langle rule \rangle, \langle statement \rangle, \langle filter_item \rangle, \langle entity \rangle, \langle attribute \rangle, \langle operator \rangle, \langle value \rangle\}$.
- R is the set of production rules of the form $A ::= \alpha$, defined explicitly by the BNF notation below. Each rule specifies that a non-terminal $A \in N$ can be replaced by a string $\alpha \in (\Sigma \cup N)^*$.
- $S = \langle ruleset \rangle$ is the start symbol.

We define the production rules below, where ϵ denotes the empty string, used to represent unspecified filtering components:

$$\begin{aligned}
 \langle ruleset \rangle &::= \langle rule \rangle \mid (\langle ruleset \rangle AND \langle ruleset \rangle) \mid (\langle ruleset \rangle OR \langle ruleset \rangle) \\
 \langle rule \rangle &::= INCLUDE \langle statement \rangle \mid EXCLUDE \langle statement \rangle \\
 &\quad \mid INCLUDE \langle statement \rangle AND EXCLUDE \langle statement \rangle \\
 \langle statement \rangle &::= \{ \langle filter_item \rangle (, \langle filter_item \rangle)^* \} \\
 \langle filter_item \rangle &::= (\langle entity \rangle, \epsilon, \epsilon, \epsilon) \mid (\langle entity \rangle, \langle attribute \rangle, \langle operator \rangle, \langle value \rangle) \\
 \langle entity \rangle &::= \text{an item} \in \mathbb{U}_{otype} \cup \mathbb{U}_{etype} \\
 \langle attribute \rangle &::= \text{an attribute} \in \mathbb{U}_{att} \\
 \langle operator \rangle &::= < \mid > \mid = \mid \neq \mid \dots \\
 \langle value \rangle &::= \text{a value} \in \mathbb{U}_{val}.
 \end{aligned}$$

Let $L(G)$ be a language generated by this grammar, i.e., $L(G) = \{w \in \Sigma^* \mid S \xRightarrow{*} w\}$, where $S \xRightarrow{*} w$ denotes that w can be derived from the start symbol S by applying the production rules in R zero or more times. We call a query q valid if and only if $q \in L(G)$.

The list of operators can be extended beyond what we defined here to enable richer filtering. A ruleset written in this language defines the conditions under which events and objects should be included in or excluded from the scope of a process. A query is considered valid if and only if it belongs to the language defined by the grammar.

3.3 The enrichment procedure

Given a valid ruleset expressed in the the enrichment language, we apply a transformation procedure to generate an scope-enriched OCEL. This procedure evaluates the ruleset over the original log, filters relevant events and associated objects, and instantiates a new process object representing the defined scope. The newly created process object is assigned a unique identifier and linked to all selected entities through appropriate OCEL relations. The result is a scope-enriched OCEL in which the process scope is explicitly represented and structurally embedded in compliance with the OCEL schema.

We do not formalize the detailed operational semantics of the enrichment process, as the transformation is deterministic and straightforward once the ruleset is defined. Instead, the remainder of the paper focuses on demonstrating how scope-enriched OCELS enable systematic discovery and visualization of inter-process interoperability, supported by tools and artifacts implemented around the proposed language and its semantics - as a proof of concept.

4 Implementation

We developed two complementary tools to support the proposed approach. The first, **Procellar**, enables analysts to define rulesets based on which they export an scope-enriched OCEL from an existing OCEL. The second, **Business Execution Graph**, facilitates the analysis of process interactions based on scope-enriched OCEL. Both tools feature front-end and back-end components, implemented in JavaScript and Python, respectively.

4.1 Procellar

Procellar allows users to import an OCEL log and define process scope rulesets through an interactive interface. Each ruleset consists of one or more rules, combined using logical operators as defined by the grammar of the enrichment language. Once defined, rulesets can be applied to enrich the OCEL, resulting in a scope-enriched version of the log. Both the scope-enriched OCEL and the corresponding rulesets can be exported for further analysis or reuse.

The tool provides two modes for rule definition: a *basic mode* and an *advanced mode*. In basic mode, users define rules by including or excluding specific object types without filtering attribute values. This mode simplifies the scoping process when fine-grained filtering is unnecessary. In contrast, advanced mode allows the specification of complex scoping criteria involving combinations of event or object attributes.

Fig. 2 illustrates the rule definition interface in advanced mode. The area marked by (1) shows the rule construction panel, where users can select the entity type (object or

Fig. 2: Rule definition for enabling scope-enriched OCEL with process notion in **Procellar**, where area marked with (1) represents a filter item, and area marked with (2) represents implementation adjustment for better usability for including object types or event types without any filter.

event), choose attributes, specify operators, and enter corresponding values. The area marked by (2) provides a shortcut for including object and event types, i.e., activities, without attribute filtering, which automatically generates rules with empty filtering conditions as defined in the grammar. Procellar is available as an open-source project on GitHub⁵.

4.2 Business Execution Graph (BEG)

The Business Execution Graph (BEG) tool enables the analysis and visualization of interactions among processes in a scope-enriched OCEL. It supports the identification of shared objects, dependencies, and potential coordination structures between processes.

Users can configure the visualization through various settings. Node sizes can be adjusted based on object counts or object type diversity. Edge labels can be customized to display object types, total shared objects, or average flow times, enabling analysts to explore process dependencies considering different measures. For example, labeling edges by object type allows investigation of how different object types are involved in process interactions.

In addition, node colors can be dynamically styled based on the number of incoming, outgoing, or total edges. The network visualization can be exported in PNG format for reporting or presentation purposes. The tool also supports exporting the graph to the VOSviewer format for advanced network analysis and clustering. Business Execution Graph is also available as an open-source tool on GitHub⁶.

5 Demonstration

To demonstrate the applicability and flexibility of our approach, we applied the proposed toolsets to a publicly available Object-Centric Event Log (OCEL) from the lo-

⁵<https://github.com/hudsonjychen/procellar>

⁶<https://github.com/hudsonjychen/business-execution-graph>

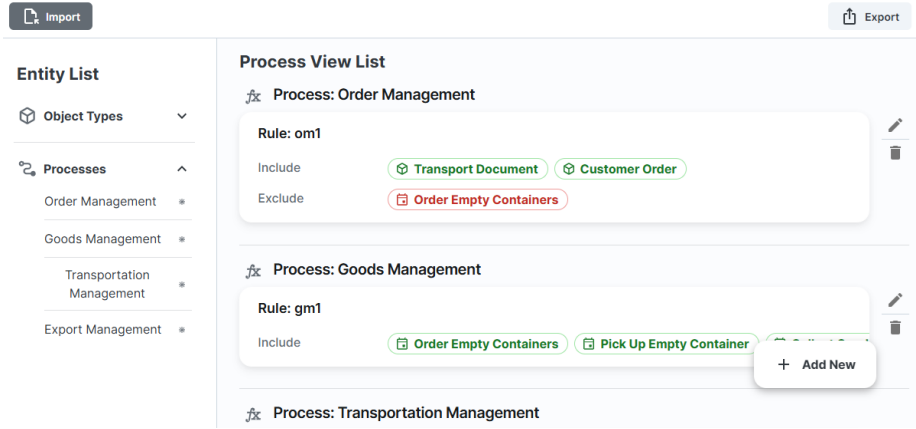


Fig. 3: Rule configuration in **Procellar** for projecting logistics processes

logistics domain⁷. This event log captures an end-to-end logistics scenario, beginning with customer order placement and concluding with the transportation of goods via various long-distance carriers, such as ships and trains. It involves interactions among multiple object types, including orders, goods, transport documents, and containers. We applied our enrichment methodology by defining four distinct but interrelated processes (through 4 defined rules), each representing a key phase in the logistics value chain:

- *Order Management*: Includes events related to the creation and management of customer orders, concluding with the issue of transport documents needed for starting the goods management process (594 events).
- *Goods Management*: Covers preparatory activities, including ordering empty containers, collecting goods, and loading goods into trucks, making them ready for dispatch to terminals (13155 events).
- *Transportation Management*: Focuses on the movement of containers using ground vehicles between warehouses and terminals, capturing logistical transport operations before long-distance carriers' activities (18314 events).
- *Export Management*: Represents the final phase, where containers are scheduled and loaded onto ships or trains for long-distance export (2132).

Fig. 3 shows a screenshot from the Procellar tool, illustrating how the process scopes were defined using a combination of object types and event filtering. Each process was specified through a corresponding ruleset, allowing the log to be modularly partitioned and enriched. The resulting scope-enriched OCEL and the projection rulesets are publicly available in our GitHub repository, enabling reproducibility and further analysis by the research community⁸.

The scope-enriched OCEL served as input to the Business Execution Graph (BEG), which was used to visualize interactions and dependencies between the defined processes. Fig. 4 presents the resulting graph, where each node represents a process and

⁷<https://ocel-standard.org/event-logs/simulations/logistics/>

⁸<https://github.com/shahrzadkayatbashi/Process-Level-OCPEM>

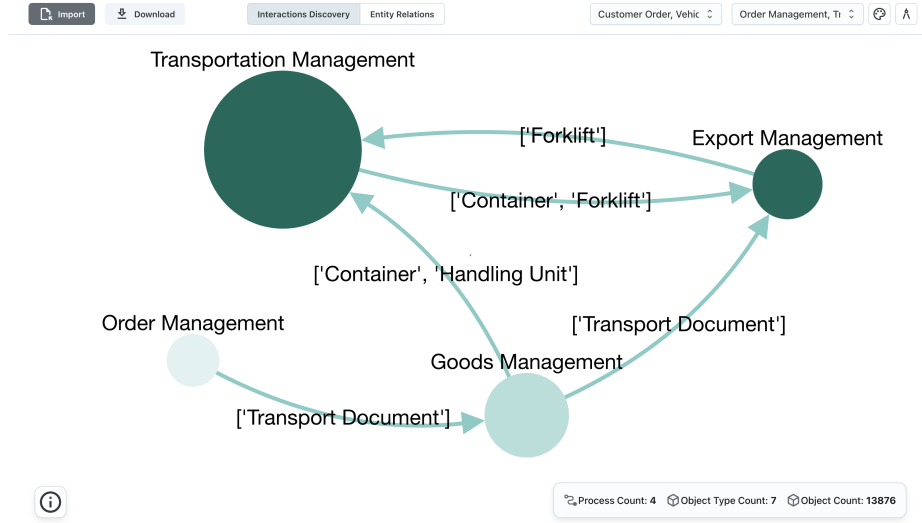


Fig. 4: **BEG** visualization of interactions among logistics processes

edges capture object types that are handed over between processes. Node sizes are proportional to the number of associated object instances, while edge labels denote the object types that facilitate transitions. Node colors indicate the number of incoming edges, providing visual clues about process centrality and interconnectivity.

As the figure illustrates, the *Order Management* process initiates the chain by transmitting *Transport Document* objects to the *Goods Management* process. The latter, in turn, provides inputs for both the *Transportation Management* and *Export Management* processes. Notably, the execution of *Transportation Management* relies on coordination involving both *Container* and *Handling Unit* object types. The *Export Management* process depends on the outcomes of *Transportation Management*, especially in the use of *Forklifts* to transfer containers to export terminals. A notable pattern observed in the graph is a loop, indicating forklifts returning empty after container delivery.

This demonstration validates the effectiveness of our approach in handling object-centric event data. By enabling structured projection into modular process scopes and providing visual insights into process interactions, our toolset supports a systematic analysis of complex, interconnected operations. The resulting scope-enriched OCEL and interaction graph can facilitate a deeper understanding of process dependencies, handovers, and coordination mechanisms within object-rich domains.

6 Conclusion

This paper addresses a key limitation in Object-Centric Process Mining (OCPM), namely the absence of explicit process scope representations in current object-centric event data standards such as OCEL 2.0. To overcome this challenge, we proposed a scope-enriched framework that allows analysts to define and embed process scopes directly

within OCEL. Our contribution includes a formal grammar-based language for specifying scoping rules and two open-source tools (Procellar and Business Execution Graph) that operationalize this approach.

By enabling the systematic construction of scope-enriched OCELs, our framework bridges the gap between data-driven analysis and domain-specific process knowledge. The introduced tools support modular process projection, scope-aware event filtering, and the visualization of inter-process dependencies based on object-centric interactions. Our demonstration using a public logistics dataset illustrates how the approach facilitates the discovery of process boundaries, overlaps, and coordination patterns in complex operational environments. The proposed method enhances the expressiveness and interpretability of OCPM, offering analysts greater control over the level of abstraction and enabling structured reasoning about process architecture. It supports both fine-grained analysis and high-level process aggregation, making it suitable for a wide range of analytical and organizational needs.

Future work may focus on extending the language to support temporal constraints and behavioral patterns within scopes, exploring semi-automatic discovery approaches to support analyst-defined rules, and integrating the approach into enterprise modeling environments to support strategic alignment and process governance. In addition, we aim to apply this approach in a real case, where we can evaluate how a scope-enriched approach would be perceived by analysts, and can eventually improve the overall Object-Centric Process Mining experience and adaptation in practice.

References

1. W.M.P. van der Aalst. Object-centric process mining: Dealing with divergence and convergence in event data. In *Software Engineering and Formal Methods*, pages 3–25. Springer, 2019.
2. Shahrzad Khayatbashi, Viktor Sjölin, Anders Granåker, and Amin Jalali. AI-enhanced business process automation: a case study in the insurance domain using object-centric process mining. In *International Conference on Business Process Modeling, Development and Support (BPMDS)*, pages 3–18. Springer, 2025.
3. Alessandro Berti, Urszula Jessen, Gyunam Park, Majid Rafiei, and W.M.P. van der Aalst. Analyzing interconnected processes: using object-centric process mining to analyze procurement processes. *International Journal of Data Science and Analytics*, pages 1–23, 2023.
4. Dina Kretzschmann, Gyunam Park, Alessandro Berti, and W.M.P. van der Aalst. Overstock problems in a purchase-to-pay process: An object-centric process mining case study. In *Advanced Information Systems Engineering Workshops*, pages 347–359. Springer, 2024.
5. Gyunam Park, Sevede Aydin, Cüneyt Uğur, and W.M.P. van der Aalst. Analyzing an after-sales service process using object-centric process mining: A case study. In *Process Mining Workshops*, pages 406–418. Springer, 2023.
6. W.M.P. van der Aalst. Academic view: development of the process mining discipline. In *Process Mining in Action: Principles, Use Cases and Outlook*, pages 181–196. Springer, 2020.
7. Dirk Fahland, Marco Montali, Julian Leberherz, W.M.P. van der Aalst, Maarten van Asseldonk, Peter Blank, Lien Bosmans, Marcus Brenscheidt, Claudio di Ciccio, Andrea Delgado, et al. Towards a simple and extensible standard for object-centric event data (oced)-core model, design space, and lessons learned. *arXiv preprint arXiv:2410.14495*, 2024.

8. Stefan Esser and Dirk Fahland. Multi-dimensional event data in graph databases. *Journal on Data Semantics*, 10(1):109–141, 2021.
9. Shahrzad Khayatbashi, Olaf Hartig, and Amin Jalali. Transforming object-centric event logs to temporal event knowledge graphs. In *Business Process Management Workshops: Object-centric Processes from A to Z (OBJECTS)*, pages 300–313. Springer, 2024.
10. Alessandro Berti, István Koren, Jan Niklas Adams, Gyunam Park, Benedikt Knopp, Nina Graves, Majid Rafiei, Lukas Liß, Leah Tacke Genannt Unterberg, Yisong Zhang, Christopher Schwanen, Marco Pegoraro, and W.M.P. van der Aalst. OCEL (Object-Centric Event Log) 2.0 specification, 2023.
11. W.M.P. van der Aalst and Alessandro Berti. Discovering object-centric petri nets. *Fundamenta informaticae*, 175(1-4):1–40, 2020.
12. Lukas Liss, Caspar Mensing, and W.M.P. van der Aalst. Object-centric causal nets. In *International Conference on Advanced Information Systems Engineering (CAiSE)*, pages 94–110. Springer, 2025.
13. Jan Niklas Adams and W.M.P. van der Aalst. Precision and fitness in object-centric process mining. In *2021 3rd International Conference on Process Mining (ICPM)*, pages 128–135. IEEE, 2021.
14. Shahrzad Khayatbashi, Najmeh Miri, and Amin Jalali. OLAP operations for object-centric process mining. In *International Conference on Advanced Information Systems Engineering (CAiSE Forum)*, pages 111–118. Springer, 2025.
15. Amin Jalali. Object type clustering using markov directly-follow multigraph in object-centric process mining. *IEEE Access*, 10:126569–126579, 2022.
16. Jan Niklas Adams, Gyunam Park, and W.M.P. van der Aalst. ocpa: A python library for object-centric process analysis. *Software Impacts*, 14:100438, 2022.
17. Marlon Dumas, L Marcello Rosa, Jan Mendling, and A Hajo Reijers. *Fundamentals of business process management*. Springer, 2018.
18. Mathias Weske. Business process management: Concepts, languages, architectures. In *Business process management: Concepts, languages, architectures*, pages 333–371. Springer, 2012.
19. W.M.P. van der Aalst. Process-oriented architectures for electronic commerce and interorganizational workflow. *Information systems*, 24(8):639–671, 1999.
20. W.M.P. van der Aalst. Intra-and inter-organizational process mining: Discovering processes within and between organizations. In *IFIP Working Conference on The Practice of Enterprise Modeling (POEM)*, pages 1–11. Springer, 2011.
21. W.M.P. van der Aalst. Twin transitions powered by event data. using object-centric process mining to make processes digital and sustainable. In *Workshop on Algorithms & Theories for the Analysis of Event Data (ATAED)*, 2023.
22. Shahrzad Khayatbashi, Najmeh Miri, and Amin Jalali. Advancing object-centric process mining with multi-dimensional data operations. *arXiv preprint arXiv:2412.00393*, 2024.